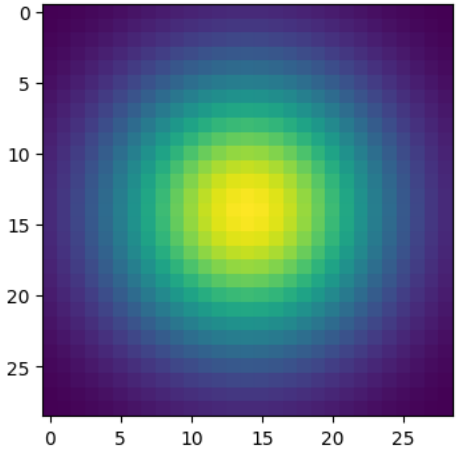
Part 1: Image filtering

Insert visualization of Gaussian kernel from project-1.ipynb here:

1D:

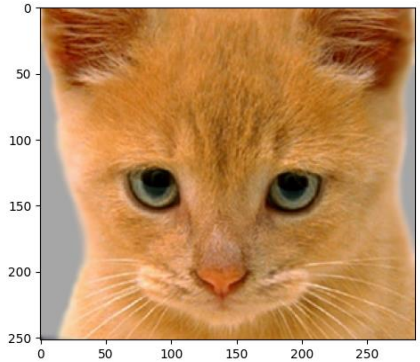
2D:



The **my_conv2d_numpy()** function uses 2D convolution by applying a filter to each channel of an image by itself. First, the image is zero-padded by adding rows and columns of zeros around its borders, with the padding amount determined by half the filter size to make sure the output dimensions equal the input. For each color channel & pixel position, the function gets a local neighborhood from the padded image to match the filter size to make the output pixel value, multiplies between the image patch and filter, and sums the resulting values to produce a single output pixel value.. This is repeated for every pixel location.

Part 1: Image filtering

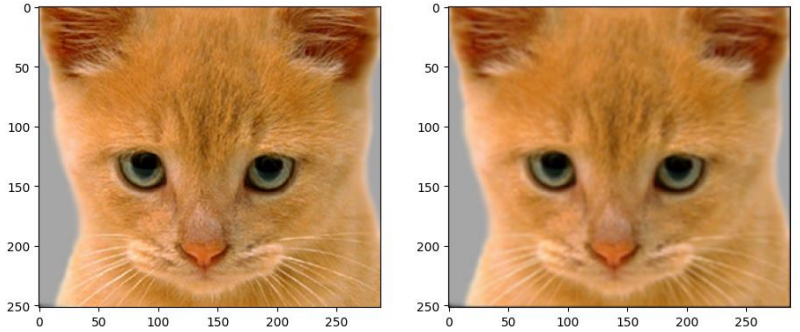
Identity filter



Briefly Explain what this filter did:

This filter preserves the original image and assigns full weight to the center pixel and zero weight otherwise, resulting in no visible change.

Small blur with a box filter

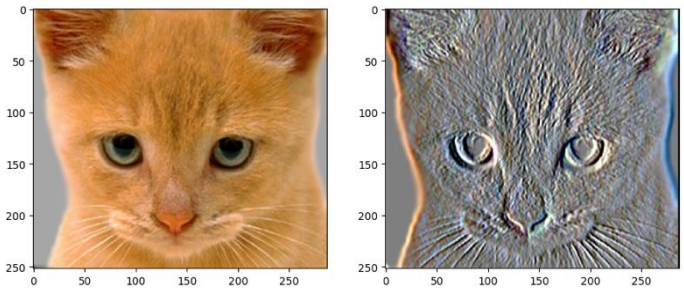


Briefly Explain what this filter did:

The box filter creates slight smooth edges and a mild blur.

Part 1: Image filtering

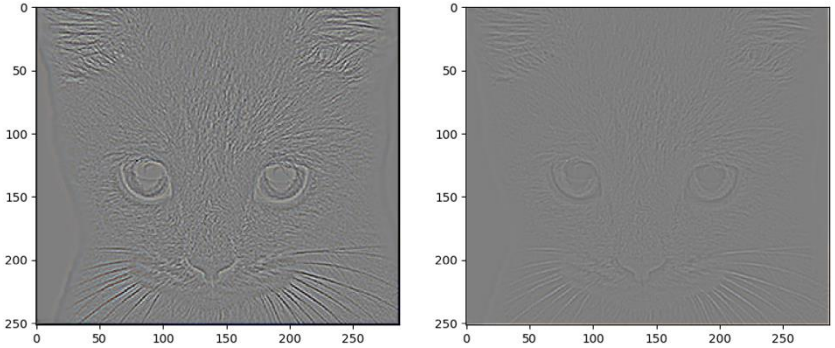
Sobel filter



Briefly Explain what this filter did:

This filter highlights the image gradients and emphasizes edges by detecting intensity changes in a specific direction, making the edges more prominent and suppresses the uniform regions

Discrete Laplacian filter



Briefly Explain what this filter did:

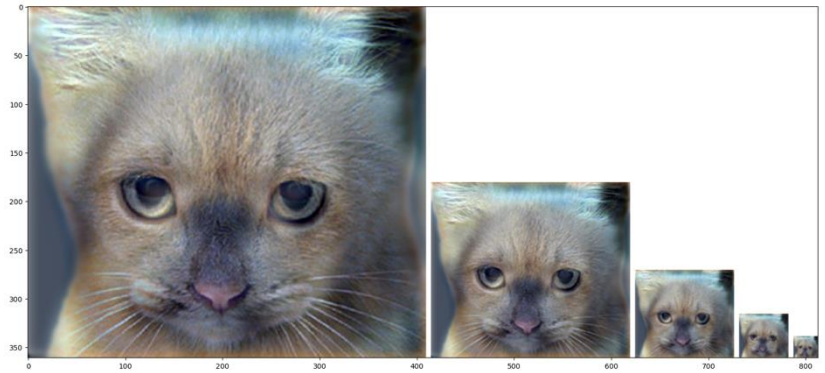
This filter detects edges by responding to the rapid changes in all directions, making strong edge responses and highlighting the fine details.

Part 1: Hybrid images

Describe the three main steps of create_hybrid_image() here. Explain how to ensure the output values are within the appropriate range for matplotlib visualizations.

The first step is to create a low-pass filter version of an image with a Gaussian filter. The second step is to get the high frequency components by subtracting the low pass filtered version from the original version. The third step is to add the two components and clip them to make sure the pixel values are valid.

Cat + Dog



Cutoff frequency: 7

Part 1: Hybrid images

Motorcycle + Bicycle

Cutoff frequency: 7

Plane + Bird

Cutoff frequency: 7

Part 1: Hybrid images

Einstein + Marilyn

Cutoff frequency: 7

Submarine + Fish

Cutoff frequency: 8

Part 2: Hybrid images with PyTorch

Cat + Dog

Motorcycle + Bicycle

Part 2: Hybrid images with PyTorch

Plane + Bird

Einstein + Marilyn

Part 2: Hybrid images with PyTorch

Submarine + Fish

Part 1 vs. Part 2

Part 1 is 6.741 seconds and Part 2 is 0.209 seconds, so part 2 is faster.

Part 3: Understanding input/output shapes in PyTorch

Consider a 1-channel 5x5 image and a 3x3 filter. What are the output dimensions of a convolution with the following parameters?

Stride = 1, padding = 0 is 3x3
- (5-3+0) / 1 + 1

Stride = 2, padding = 0 is 2x2
- (5-3+0) / 2 + 1

Stride = 1, padding = 1 is 5x5
- (5-3+2) / 1 + 1

Stride = 2, padding = 1 is 3x3
- (5-3+2) / 2 + 1

What are the input & output dimensions of the convolutions of the dog image and a 3x3 filter with the following parameters:

Stride = 1, padding = 0 is (1, 3, 359, 408)
- Height: (361 – 3 + 0) / 1 + 1 = 359
- Width: (410 – 3 + 0) / 1 + 1 = 408

Stride = 2, padding = 0 is (1, 3, 180, 204)
- Height: (361 – 3 + 0) / 2 + 1 = 180
- Width: (410 – 3 + 0) / 2 + 1 = 204

Stride = 1, padding = 1 is (1, 3, 361, 410)
- Height: (361 – 3 + 2) / 1 + 1 = 361
- Width: (410 – 3 + 2) / 1 + 1 = 410

Stride = 2, padding = 1 is (1, 3, 181, 205)
- Height: (361 – 3 + 2) / 2 + 1 = 181
- Width: (410 – 3 + 2) / 2 + 1 = 205

Part 3: Understanding input/output shapes in PyTorch

How many filters did we apply to the dog image?

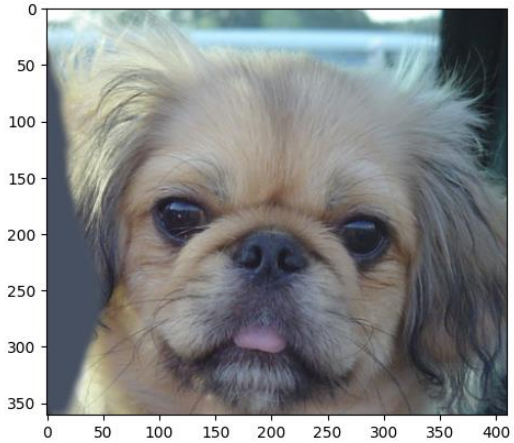
We applied one filter per output channel, since the convolution layer has out_channels as 1, a single filter was applied across all of the input channels, producing one output feature map.

Section 3 of the handout gives equations to calculate output dimensions given filter size, stride, and padding. What is the intuition behind this equation?

The equation calculates how many positions the filter occupies as it slides across the image. The input size minus the kernel size shows the range the filter ca move without falling off the image edges and the padding multiplied by 2 extends the range by adding more pixels on both sides. When divided by stride, it skips every other positions, and adding one adds the initial starting position.

Part 3: Understanding input/output shapes in PyTorch

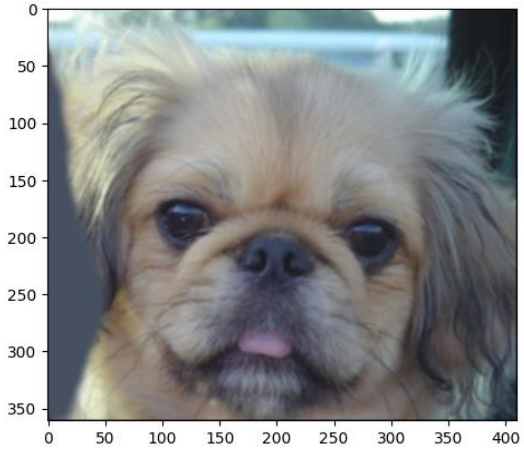
Visualization 0:



What filter was applied here?

- Identity

Visualization 1:

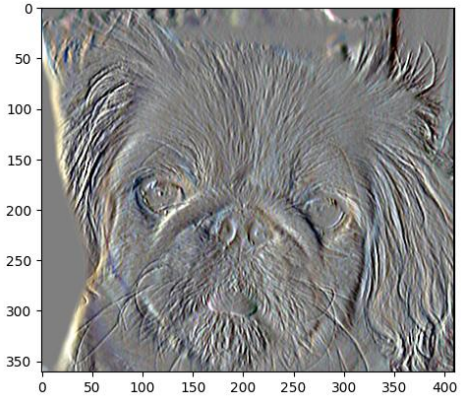


What filter was applied here?

- Blur

Part 3: Understanding input/output shapes in PyTorch

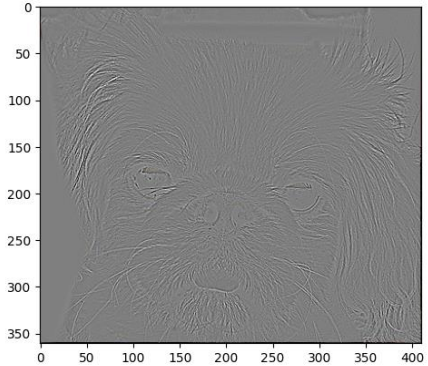
Visualization 2:



What filter was applied here?

- Sobel

Visualization 3:

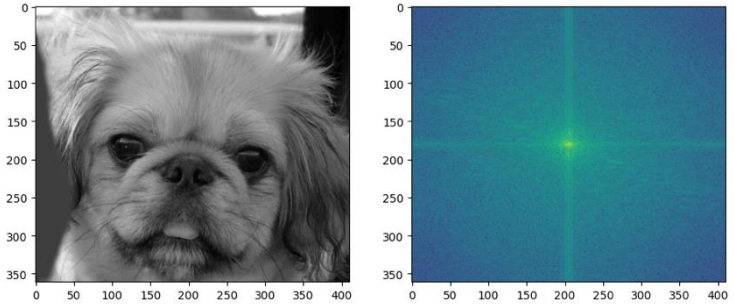


What filter was applied here?

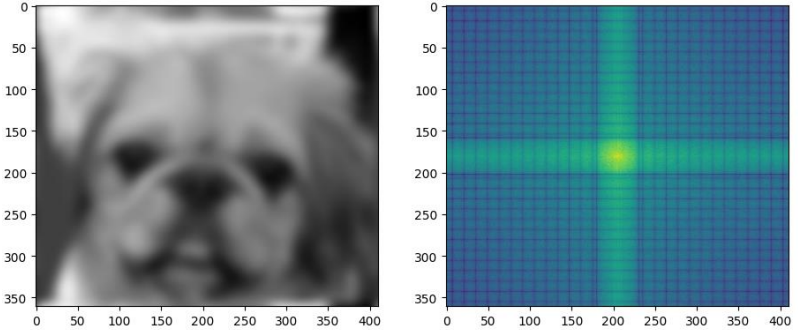
- Laplacian

Part 4: Frequency Domain Convolutions (extra credit)

[Insert the visualizations of the dog image in the spatial and frequency domain]



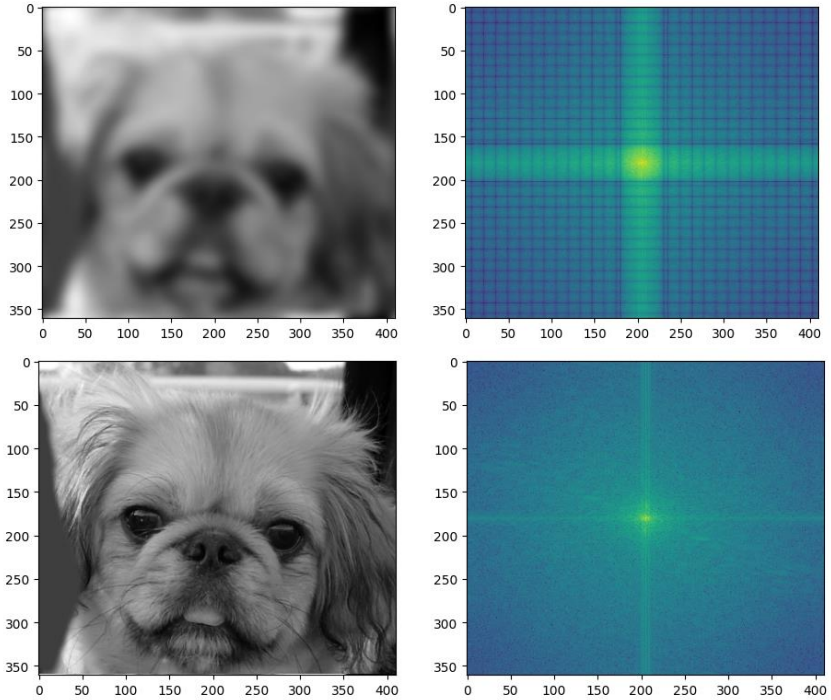
[Insert the visualizations of the blurred dog image in the spatial and frequency domain]



Explain the difference in the frequency domain visualizations for the dog images:
Blurring ends up suppressing the high frequency, concentrating energy near the center

Part 4: Frequency Domain Convolutions (extra credit)

Insert the visualizations frequency of the 2D Gaussian in the spatial and domain



Try out some different cutoff values for the 2D Gaussian. What relationship do you notice between the cutoff value and the frequency domain? Why is that?

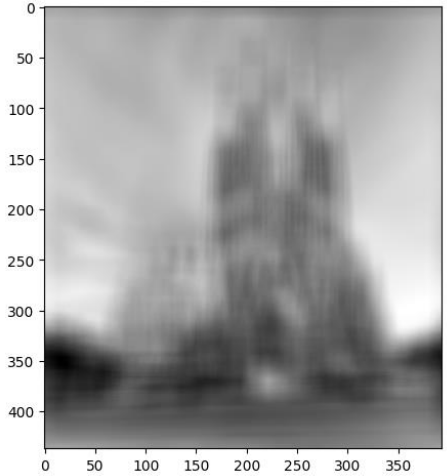
A larger cutoff value has a wider Gaussian and a narrower frequency response. A smaller cutoff has a sharper Gaussian and a wider frequency spread.

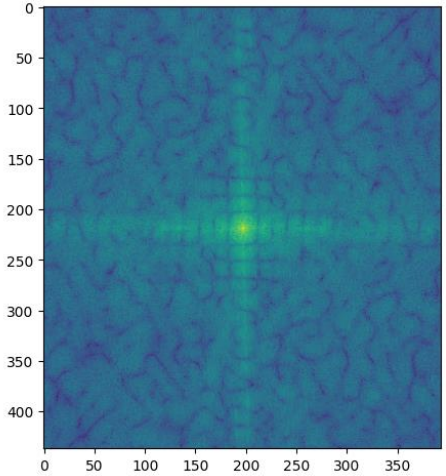
Part 4: Frequency Domain Convolutions (extra credit)

Briefly explain the Convolution Theorem and why this is related to deconvolution.
The convolution theorem is multiplication in frequency domain and deconvolution is division, which is unstable when frequencies are near zero.

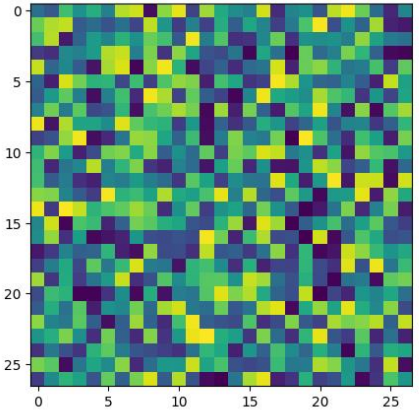
Part 4: Frequency Domain Convolutions (extra credit)

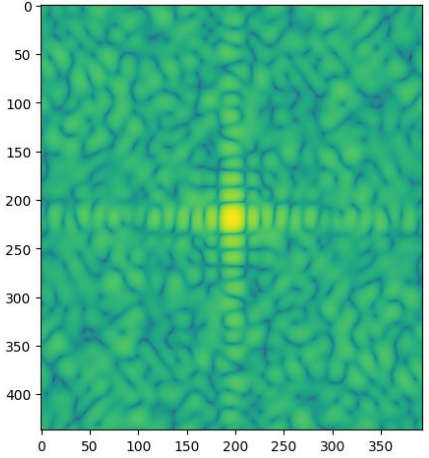
Insert the visualizations of the mystery image in the spatial and frequency domain.





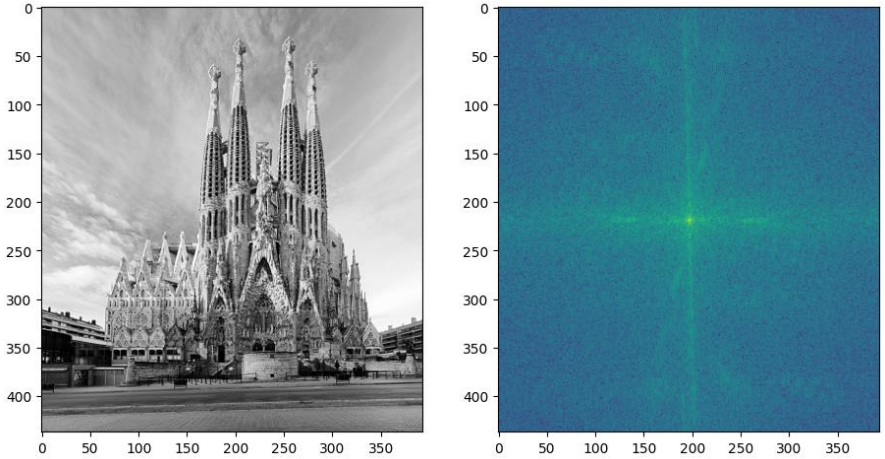
Insert the visualizations of the mystery kernel in the spatial and frequency domain.



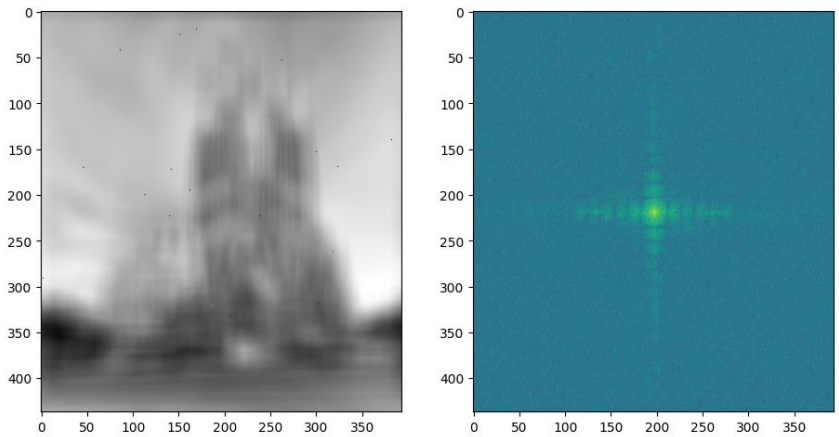


Part 4: Frequency Domain Convolutions (extra credit)

Insert the de-blurred mystery image and its visualizations in the spatial and frequency domain



Insert the de-blurred mystery image and its visualizations in the spatial and frequency domain after adding salt and pepper noise



Part 4: Frequency Domain Convolutions (extra credit)

What factors limit the potential uses of deconvolution in the real world? Give two possible factors

One factor is that deconvolution is sensitive to noise, since dividing by small values in the frequency domain amplifies small amounts of noise. Another factor is that deconvolution needs precise knowledge of the blur kernel, but in the real world, the kernel is usually unknown leading to unstable reconstruction of the image.

Describe any structures found in the frequency domain of the mystery image and explain what it's caused by.

The frequency domain of the mystery image shows a strong central peak and cross-shaped structures along the horizontal and vertical axes. The bright center shows the dominant low frequency content in the blurred image, and the cross patterns comes from the directional nature of the blur kernel and padding effects during convolution.

Conclusion

How does varying the cutoff frequency value or swapping images within a pair influences the resulting hybrid image?

Varying the cutoff frequency controls how much low and high frequency information is preserved in the hybrid image. A lower cutoff frequency makes stronger blurring, while a higher cutoff frequency has more fine details and makes it more visible. Swapping the images within a pair changes which image contributes the low frequency structure versus the high frequency details, affecting the perception. Hybrid works when the low frequency image has strong structure and the high frequency image has sharp edges.